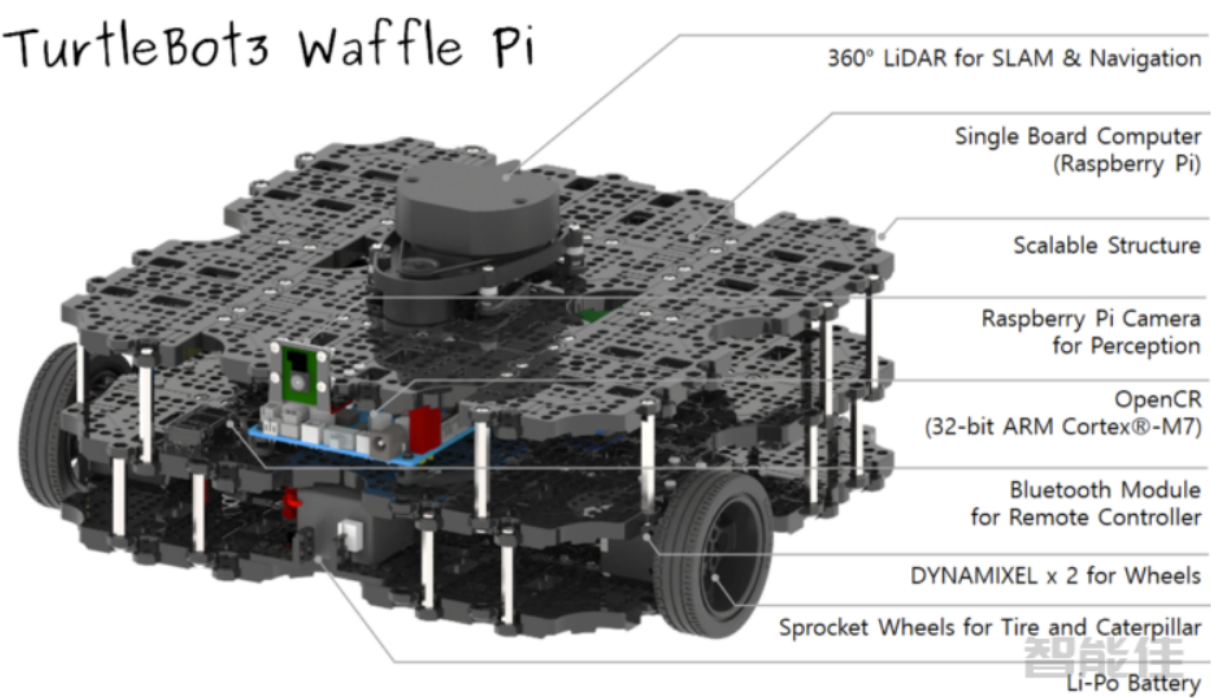


1. TurtleBot3 URDF建模详解



1.1 机器人型号对比

特性	Burger	Waffle	Waffle Pi
激光雷达	✓	✓	✓
IMU	✓	✓	✓
摄像头	X	X	✓
深度相机	X	✓ (可选)	✓
尺寸	小型	中型	中型

1.2 URDF结构分析

/opt/ros/jazzy/share/turtlebot3_gazebo/urdf/turtlebot3_waffle.urdf

主要组成部分：

1. **base_footprint**: 机器人在地面的投影点
2. **base_link**: 机器人主体
3. **wheel_left_link/wheel_right_link**: 左右驱动轮
4. **caster_back_left_link/caster_back_right_link**: 后方支撑轮
5. **imu_link**: IMU传感器
6. **base_scan**: 激光雷达

7. camera_link (仅Waffle Pi): 摄像头系统

关键参数:

```
<!-- 轮子配置 -->
<origin xyz="0.0 0.144 0.023" rpy="-1.57 0 0"/> <!-- 轮距0.288m -->
<cylinder length="0.018" radius="0.033"/> <!-- 轮子半径3.3cm -->

<!-- 激光雷达位置 -->
<origin xyz="-0.064 0 0.122" rpy="0 0 0"/> <!-- 高度12.2cm -->

<!-- IMU位置 -->
<origin xyz="0.0 0 0.068" rpy="0 0 0"/> <!-- 高度6.8cm -->
```

1.3 Waffle vs Waffle Pi 主要差异

Waffle Pi相比Waffle的额外组件:

```
<!-- 摄像头系统 (仅Waffle Pi) -->
<joint name="camera_joint" type="fixed">
  <origin xyz="0.073 -0.011 0.084" rpy="0 0 0"/>
  <parent link="base_link"/>
  <child link="camera_link"/>
</joint>
<joint name="camera_rgb_joint" type="fixed">
  <origin xyz="0.003 0.011 0.009" rpy="0 0 0"/>
  <parent link="camera_link"/>
  <child link="camera_rgb_frame"/>
</joint>
<joint name="camera_rgb_optical_joint" type="fixed">
  <origin xyz="0 0 0" rpy="-1.57 0 -1.57"/>
  <parent link="camera_rgb_frame"/>
  <child link="camera_rgb_optical_frame"/>
</joint>
```

2. TurtleBot3 控制方式

2.1 基础控制接口

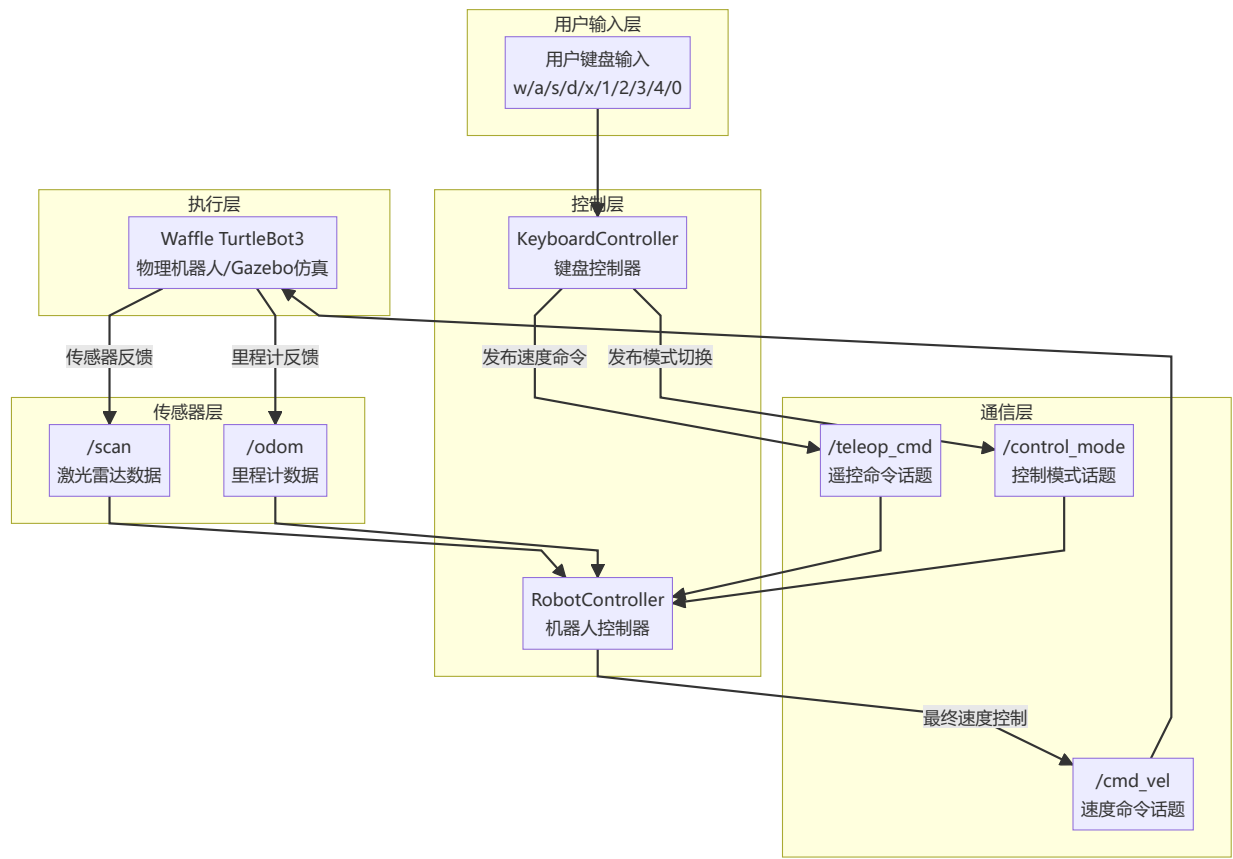
主要话题:

- `/cmd_vel` (geometry_msgs/Twist): 速度控制命令
- `/scan` (sensor_msgs/LaserScan): 激光雷达数据
- `/imu` (sensor_msgs/Imu): IMU数据
- `/image_raw` (sensor_msgs/Image): 摄像头图像 (仅Waffle Pi)
- `/odom` (nav_msgs/Odometry): 里程计数据

3 TurtleBot3 控制框架

KeyboardController、RobotController和waffle小车的详细关系。

3.1 系统架构图



3.2 组件详细关系

3.2.1 KeyboardController (键盘控制器)

功能角色：用户输入接口层

- **主要职责：**
 - 捕获键盘输入 (`w/a/s/d/x` 用于运动控制, `1/2/3/4/0` 用于模式切换)
 - 将键盘输入转换为速度命令和控制模式
 - 发布到ROS话题供其他节点使用
- **发布话题：**

```
/teleop_cmd (geometry_msgs::msg::TwistStamped) // 速度命令
/control_mode (std_msgs::msg::String) // 控制模式
```

3.2.2 RobotController (机器人控制器)

功能角色：核心控制逻辑层

- 主要职责：
 - 接收键盘控制器的指令
 - 集成传感器数据（激光雷达、里程计）
 - 实现多种控制模式（手动、自动前进、圆形运动、避障、墙面跟随）
 - 安全监控和紧急停止
 - 最终速度命令输出
- 订阅话题：

```
/teleop_cmd      // 来自KeyboardController的遥控命令  
/control_mode    // 来自KeyboardController的模式切换  
/scan            // 激光雷达数据  
/odom            // 里程计数据
```

- 发布话题：

```
/cmd_vel (geometry_msgs::msg::Twist/TwistStamped) // 最终速度命令
```

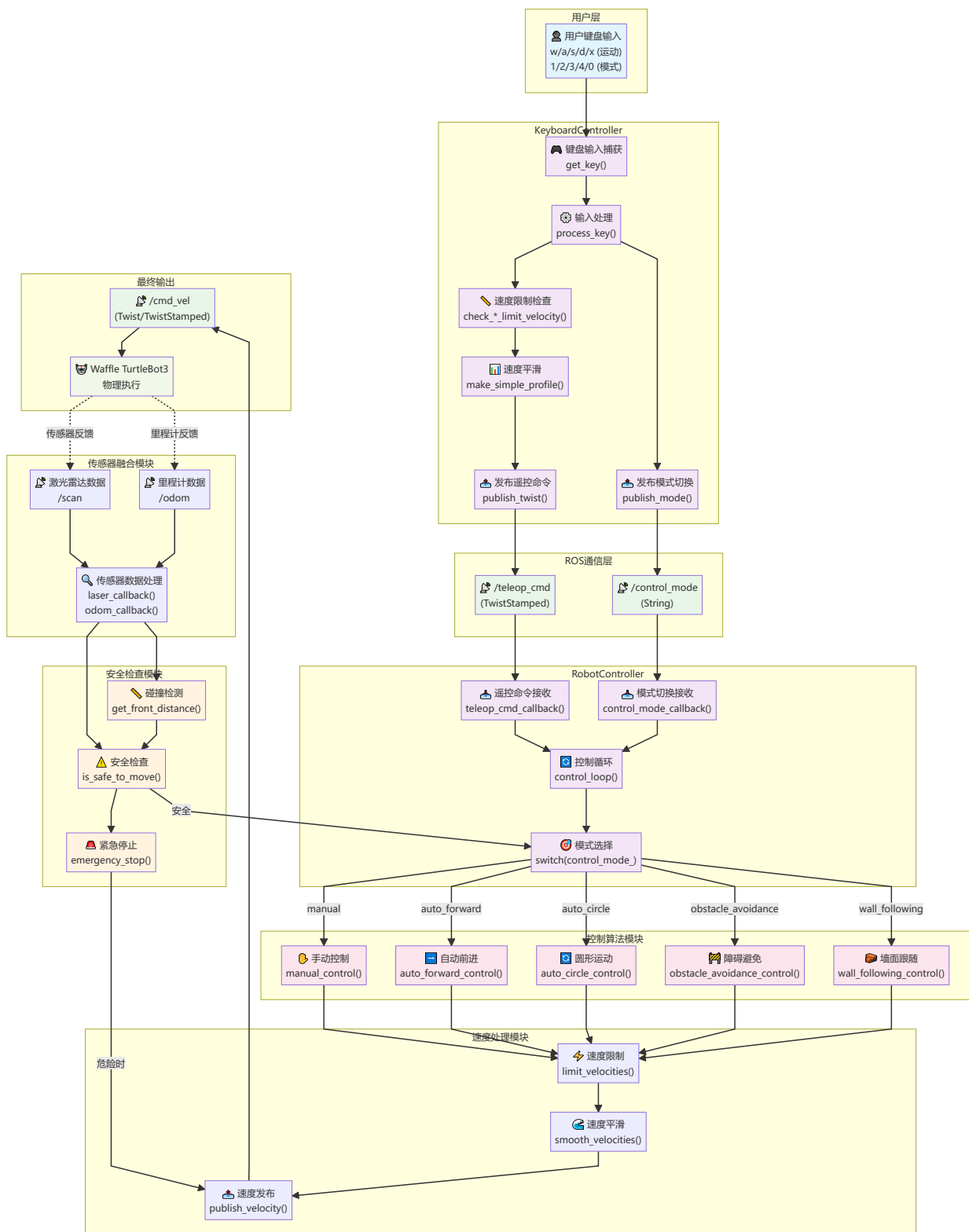
3.2.3 Waffle TurtleBot3

功能角色：物理执行层

- 硬件特性 (Waffle模型)：

```
最大线速度：0.26 m/s  
最大角速度：1.82 rad/s  
传感器：LDS激光雷达、IMU、编码器
```

3.2.4 数据流向分析



3.3 详细设计流程

3.3.1 第一阶段：KeyboardController设计

1. 输入处理模块

```
// 核心方法
char get_key()                // 非阻塞键盘输入捕获
bool process_key(char key)    // 键盘输入处理和映射
```

1. 速度控制模块

```
// 速度限制（基于waffle模型）
double check_linear_limit_velocity(double velocity) // 线速度限制
double check_angular_limit_velocity(double velocity) // 角速度限制
double make_simple_profile(...) // 速度平滑算法
```

1. 模式发布模块

```
void publish_mode(const std::string& mode) // 发布控制模式
void publish_twist() // 发布速度命令
```

3.3.2 第二阶段：RobotController设计

1. 数据接收模块

```
void teleop_cmd_callback(...) // 接收键盘控制命令
void laser_callback(...) // 接收激光雷达数据
void odom_callback(...) // 接收里程计数据
void control_mode_callback(...) // 接收模式切换命令
```

1. 控制算法模块

```
// 多种控制模式实现
void manual_control() // 手动控制模式
void auto_forward_control() // 自动前进模式
void auto_circle_control() // 圆形运动模式
void obstacle_avoidance_control() // 障碍物避免模式
void wall_following_control() // 墙面跟随模式（PID控制）
```

1. 安全监控模块

```
bool is_safe_to_move() // 安全检查
void emergency_stop() // 紧急停止
double get_front_distance() // 前方距离检测
```

1. 速度处理模块

```
void limit_velocities()           // 速度限制和加速度控制
void smooth_velocities()         // 速度平滑处理
void publish_velocity()          // 最终速度发布
```

3.4 系统运行流程

3.4.1 初始化阶段

```
// KeyboardController启动
- 检测TurtleBot3模型类型 (burger/waffle)
- 检测ROS版本 (确定使用Twist还是TwistStamped)
- 初始化发布者和定时器
- 设置终端为非规范模式

// RobotController启动
- 声明参数 (速度限制、PID参数等)
- 创建订阅者和发布者
- 启动控制循环定时器
```

3.4.2 手动控制流程

1. 用户按键 → `KeyboardController::get_key()`
2. 键盘处理 → `KeyboardController::process_key()`
3. 速度计算 → `KeyboardController::check_*_limit_velocity()`
4. 发布命令 → `/teleop_cmd`话题
5. 接收命令 → `RobotController::teleop_cmd_callback()`
6. 安全检查 → `RobotController::is_safe_to_move()`
7. 速度发布 → `RobotController::publish_velocity()`
8. 小车执行 → `waffle`接收`/cmd_vel`命令

3.4.3 自动控制流程

1. 模式切换 → `KeyboardController`发布控制模式
2. 模式接收 → `RobotController::control_mode_callback()`
3. 控制循环 → `RobotController::control_loop()`
4. 传感器融合 → 激光雷达 + 里程计数据处理
5. 算法执行 → 对应的自动控制算法
6. 安全监控 → 碰撞检测和紧急避障
7. 速度输出 → 最终速度命令发布

3.5 安全机制设计

3.5.1 多层安全检查

- KeyboardController层：速度限制检查
- RobotController层：传感器安全检查
- 紧急停止机制：检测到危险立即停止

3.5.2 速度平滑算法

- 防止突然加速/减速
- 保护机器人机械结构
- 提供更平稳的运动体验

3.5.3 模式切换保护

- 模式切换时自动停止当前运动
- 防止模式冲突导致的不可预测行为